【必读】Feature 分支使用说明书

> 适用于所有参与 CSSA-Web 项目的开发成员，请在开始写第一行代码之前读完。

一、三条铁律（违反会造成大麻烦）

> ❌ 不能在 `main` 直接写代码
> ❌ 不能在 `dev` 直接写代码
> ✅ 只能在自己的 `feature` 分支写代码

二、分支结构总览

```
main          ← 正式上线版本，只有负责人才能合并进来
  └── dev     ← 团队集成测试，所有人的 feature 最终汇聚到这里
        ├── feature-jack-首页制作        ← Jack 的专属工作区
        ├── feature-lisa-登录页面        ← Lisa 的专属工作区
        └── feature-tom-网站样式美化     ← Tom 的专属工作区
```

三、分支命名规则（必须严格遵守）

格式： `feature-你的名字-你负责的功能`

正确示例	错误示例
`feature-jack-首页制作`	`jack的分支`
`feature-lisa-登录页面`	`dev-lisa`
`feature-tom-网站样式美化`	`feature`
`feature-austin-活动页面`	`mywork`

> 命名错误的分支不会被接受合并，负责人会直接要求你重新创建。

四、第一次加入项目：创建你的 Feature 分支

用 GitHub Desktop 操作（推荐新手）

1. 打开 GitHub Desktop，确认左上角显示的是 **CSSA-Web** 仓库
2. 点击顶部中间的 **Current Branch**
3. 在下拉里找到 `dev`，点击切换到 `dev`（重要：必须从 dev 创建，不能从 main）
4. 再次点击 **Current Branch** → 点击 **New Branch**
5. 输入你的分支名，例如：`feature-jack-首页制作`
6. 确认 **From** 显示的是 `dev`（如果显示 `main` 要改掉）
7. 点击 **Create Branch**
8. 点击 **Publish branch** 推送到 GitHub

五、日常工作流程（每次写代码都按这个来）

```
写代码 → 保存 → GitHub Desktop 提交 → 推送 → 发 PR → 等审核
```

第一步：写代码

在你的编辑器（VS Code 等）正常写，只要你当前在自己的 feature 分支就没问题。

第二步：在 GitHub Desktop 提交（Commit）

1. 打开 GitHub Desktop，左侧 **Changes** 会显示你改动的文件
2. 核对一下改动的文件是不是你期望的
3. 左下角 **Summary** 填写提交说明，格式建议：

   ```
   feat: 完成首页 Hero 区域布局
   fix: 修复登录按钮点击无响应
   style: 调整导航栏颜色
   ```

4. 点击 **Commit to feature-你的名字-你的功能**

第三步：推送到 GitHub（Push）

点击顶部的 **Push origin**，把代码推送到远端。

第四步：发起 Pull Request（PR）

1. 打开 github.com，进入 CSSA-Web 仓库
2. 会看到黄色横幅提示你的分支有新推送，点击 **Compare & pull request**
3. 确认方向：`feature-你的名字-xxx` → **`dev`**（不是 main！）
4. 写清楚这个 PR 做了什么，点击 **Create pull request**

5. 在群里通知负责人来审核

第五步：等待审核

- 负责人会在 PR 页面留下评论
- 如果要求修改，在自己分支改完再 commit + push，PR 会自动更新
- 审核通过后负责人会合并进 `dev`

六、常见错误与避坑指南

❌ 坑 1：在 main 或 dev 上直接写代码

症状： GitHub Desktop 左上角显示 `main` 或 `dev`，你开始改文件了。
后果： 你的改动会污染公共分支，影响所有人。
解决： 写代码之前，**永远先确认左上角显示的是自己的 feature 分支**。

❌ 坑 2：从 main 创建 feature 分支，而不是从 dev

症状： 创建分支时忘记先切到 dev，直接从 main 新建。
后果： 你的分支缺少 dev 上其他人的最新代码，合并时会产生大量冲突。
解决： 创建分支时，GitHub Desktop 的 **From** 一定要选 `dev`。

❌ 坑 3：PR 方向搞反，合并到 main 而不是 dev

症状： 发 PR 时目标分支选成了 `main`。
后果： 未经测试的代码直接进入正式上线版本。
解决： 发 PR 前，检查页面顶部的箭头方向：`你的分支` → `dev`。

❌ 坑 4：长时间不同步 dev 的最新代码

症状： 你在自己分支写了一周，dev 上已经有很多其他人的新代码。
后果： 合并时冲突一大堆，很难处理。
解决： 每隔 1-2 天，把 dev 的最新改动同步到你的分支：

1. GitHub Desktop → 切换到你的 feature 分支
2. 顶部菜单 **Branch** → **Update from dev**

❌ 坑 5：一次提交几百个文件

症状： 积累了好几天的工作，一次性全部 commit。
后果： 审核人看不懂改了什么，无法有效 review。
解决： 每完成一个小功能点就 commit 一次，每个 commit 只做一件事。

❌ 坑 6：提交了不该提交的文件

常见需要排除的文件：

- `node_modules/` 文件夹（几千个文件，不应该提交）
- `.env` / `.env.local`（包含密钥，绝对不能提交）
- `.DS_Store`（Mac 系统自动生成，没用）

解决： 这些文件应该已经在 `.gitignore` 里，如果 GitHub Desktop 里还是显示了这些文件，告诉负责人处理。

七、完整流程速查表

```
[开始新功能]
     ↓
切换到 dev 分支，拉取最新代码（Pull）
     ↓
从 dev 新建 feature-你的名字-功能名 分支
     ↓
写代码 → Commit（小步提交）→ Push
     ↓
功能完成后，发起 PR：feature → dev
     ↓
群里通知负责人审核
     ↓
按照反馈修改 → Commit → Push（PR 自动更新）
     ↓
审核通过，负责人合并进 dev ✅
     ↓
（积累足够功能且测试稳定后，负责人将 dev 合并进 main）
```

八、遇到问题怎么办

1. **先不要随便点东西**，GitHub 的很多操作不容易撤销
2. **截图当前状态**，发到群里说明情况
3. **不要自己强行解决冲突**（如果没有经验），等负责人帮你处理

> 文档维护：Austin
> 最后更新：2026-04-29